

UTC Pick Sazón

"Pide fácil, recoge con sabor"

MATERIA

Innovación y Creatividad

PROFESOR

Peña Campos Alejandro Jatniel

ALUMNOS

Alejandro Valeriano Emmanuel · Mondragón León Juan Lorenzo · Pineda Salazar Miguel Ángel

Objetivo: Crear una aplicación móvil para la gestión inteligente de pedidos en la cooperativa escolar de la Universidad Tres Culturas, permitiendo pedidos anticipados, pagos digitales o en efectivo y notificaciones en tiempo real.

Contenido

- | | |
|-------------------------------|-------------------------------------|
| 1. Resumen ejecutivo | 6. Metodología SCAMPER y evaluación |
| 2. Introducción | 7. Arquitectura del sistema |
| 3. Justificación | 8. Tecnologías utilizadas |
| 4. Objetivos | 9. Beneficios del sistema |
| 5. Planteamiento del problema | 10. Conclusiones |

Parte II — Versión extendida (documentación fuente): Anexo A. Círculo de innovación · Anexo B. Algoritmo de ejecución.

1. Resumen ejecutivo

UTC Pick Sazón es una propuesta tecnológica desarrollada para optimizar el proceso de compra y recolección de alimentos dentro de la cooperativa escolar de la Universidad Tres Culturas (UTC).

Actualmente, la cooperativa presenta problemas de saturación en horarios de alta demanda, generando largas filas, pérdida de tiempo y desorganización en la atención. Esta situación afecta directamente la experiencia de estudiantes, docentes y personal administrativo.

La solución propuesta consiste en una aplicación móvil que permite a los usuarios realizar pedidos anticipados, consultar el menú disponible, efectuar pagos digitales o en efectivo al recoger, y recibir notificaciones en tiempo real cuando su pedido esté listo.

El sistema se basa en una arquitectura cliente-servidor moderna utilizando tecnologías como Expo (React Native), NestJS, PostgreSQL, Docker y Keycloak, lo que garantiza escalabilidad, seguridad y eficiencia. Además incluye funciones innovadoras como generación de códigos, análisis de consumo y control de pedidos en tiempo real.

Ventajas que brindamos

- Pedir desde cualquier lugar.
- Pagar de forma sencilla.
- Recibir notificaciones cuando el pedido esté listo.
- Recogerlo con un código o número de pedido.
- Reducir filas y tiempos de espera innecesarios.

2. Introducción

La transformación digital ha impactado significativamente la forma en que las organizaciones ofrecen servicios. En el contexto educativo, la implementación de soluciones tecnológicas permite mejorar procesos internos y optimizar la experiencia del usuario.

En la Universidad Tres Culturas, la cooperativa escolar enfrenta problemas relacionados con la alta demanda en horarios de descanso. Esto genera largas filas, tiempos de espera elevados y una experiencia poco eficiente para los usuarios.

UTC Pick Sazón surge como una solución orientada a digitalizar el proceso de compra de alimentos mediante una aplicación móvil, permitiendo realizar pedidos desde cualquier lugar del campus, reduciendo la necesidad de hacer filas y optimizando la atención del servicio.

3. Justificación

El proyecto se justifica por la necesidad concreta de resolver la congestión que enfrenta la cooperativa escolar de la UTC durante los horarios de mayor demanda, donde las filas y los tiempos de espera afectan directamente a estudiantes, docentes y personal. La digitalización del proceso de pedido y recolección responde a un problema real y cotidiano de la comunidad universitaria.

La propuesta es pertinente y factible porque las tecnologías necesarias ya existen y son de amplio uso (aplicaciones móviles, códigos de recogida, notificaciones y pasarelas de pago). Esto permite un **costo incremental medio** y una **viabilidad media-alta**, con un **impacto alto** sobre la experiencia y la organización del servicio.

Adicionalmente, el sistema aporta valor sostenible: reduce el desperdicio de alimentos mediante la reoferta de productos preparados, genera datos de consumo que apoyan mejores decisiones de compra, e incorpora criterios de inclusión (pago en efectivo al recoger y opciones de accesibilidad) para que cualquier miembro de la comunidad pueda utilizarlo. Desde la perspectiva de la Ingeniería en Sistemas Computacionales, el proyecto integra arquitectura escalable y buenas prácticas de desarrollo, lo que lo convierte en un caso aplicable y replicable en otros planteles.

4. Objetivos

Objetivo general

Desarrollar e implementar una aplicación móvil para la gestión inteligente de pedidos en la cooperativa escolar de la Universidad Tres Culturas, que reduzca la congestión y los tiempos de espera mediante pedidos anticipados, pagos digitales o en efectivo y notificaciones en tiempo real.

Objetivos específicos

- Digitalizar el proceso de compra y recolección bajo modalidad *Pick Up* (recoger en tienda).
- Reducir las filas mediante número de pedido secuencial, pedido programado y semáforo de congestión.
- Ofrecer múltiples métodos de pago, manteniendo el efectivo como opción inclusiva.
- Notificar al usuario en tiempo real cuando su pedido esté listo para recoger.
- Proporcionar a la cooperativa herramientas de gestión: control de pedidos, inventario y datos de demanda.
- Garantizar seguridad y escalabilidad con una arquitectura cliente-servidor moderna.

5. Planteamiento del problema

El proceso actual en la cooperativa es completamente presencial, lo que genera saturación en horarios pico, tiempos de espera prolongados y desorganización en la entrega de pedidos.

Los usuarios deben acudir físicamente, realizar su pedido, pagar y esperar su preparación, lo cual provoca pérdida de tiempo y reduce la eficiencia del servicio.

Además, no existe un sistema digital que permita analizar datos de consumo, identificar productos más vendidos o gestionar la demanda de manera eficiente. Por lo tanto, se requiere una solución tecnológica que automatice el proceso de compra y mejore la organización interna de la cooperativa.

6. Metodología SCAMPER y evaluación

S Sustituir

Vamos a sustituir la manera en la que los alumnos, profesores y personal del plantel adquieren alimentos, junto con la forma en que los recogen de la cooperativa. A su vez afectará la forma en la que los dependientes registran los cobros y la entrega.

C Combinar

Combinaremos las ideas de los alumnos del ejercicio de *brainwriting* ejecutado en clase, más nuevas ocurrencias e ideas del equipo elaborador de la app.

A Adaptar

Adaptaremos los procesos de cobro y preparación que mantiene la cooperativa escolar a la idea propuesta.

M Modificar

- Se modifica el sistema de atención por filas a un sistema de pedidos digitales.
- Se atenderá según el orden de pedidos registrados en la app, no según quién llega primero a la fila.
- Se ajustará la preparación para optimizar el tiempo de entrega, priorizando pedidos ya pagados.

P Proponer

- Una app universitaria que permita pedir, pagar y recoger alimentos sin hacer fila.
- Notificaciones cuando el pedido esté listo; pago digital (tarjeta, PayPal, Mercado Pago); código o número único para recoger.
- Estadísticas de consumo para la cooperativa (productos más vendidos, horarios de mayor demanda).
- Buscamos disminuir los tiempos de espera, mejorar la organización y ofrecer una experiencia más moderna y eficiente.

E Eliminar

- Reducir las funciones del personal en el cobro para métodos como tarjeta, Mercado Pago o PayPal; el pago en efectivo mantiene su gestión.
- Reducir el espacio generado por las filas de alumnos, profesores y personal.

R Reemplazar

- El sistema manual de registro de pedidos y cobros por un sistema digital integrado en la app.
- Los tickets físicos por códigos digitales que los alumnos mostrarán al recoger.
- La dependencia exclusiva del efectivo por métodos electrónicos, manteniendo el efectivo como alternativa secundaria.

Evaluación de la propuesta

Impacto

ALTO

La implementación beneficiará a alumnos, docentes y personal administrativo al reducir significativamente los tiempos de espera en la cooperativa. Permitirá una mejor organización de los pedidos, disminuirá las filas, optimizará la preparación de alimentos y mejorará la experiencia de compra para toda la comunidad universitaria.

Viabilidad

MEDIA-ALTA

El proyecto es viable porque las tecnologías necesarias (aplicación móvil, generación de códigos y pasarelas de pago como PayPal y Mercado Pago) ya existen. Sin embargo, requiere la autorización de la institución, la colaboración de la cooperativa y la capacitación del personal para adaptarse al nuevo sistema.

Costo incremental

MEDIO

Será necesario invertir en:

- Desarrollo y mantenimiento de la aplicación.
- Integración de métodos de pago electrónicos.
- Equipo tecnológico para la cooperativa (tableta, computadora o impresora de respaldo si fuera necesaria).
- Capacitación del personal.
- Mantenimiento y actualización del sistema.

Una vez implementado, los costos de operación serían relativamente bajos en comparación con los beneficios obtenidos.

Riesgos

A MITIGAR

- Resistencia al cambio por parte del personal de la cooperativa o algunos usuarios.
- Fallas técnicas en la aplicación o problemas de conexión a Internet.
- Saturación del sistema durante horarios de alta demanda.
- Posibles inconvenientes en el procesamiento de pagos electrónicos.
- Usuarios sin teléfono inteligente o que prefieran seguir pagando en efectivo.

Estos riesgos pueden mitigarse manteniendo el sistema tradicional como alternativa durante la etapa de implementación y realizando pruebas antes del lanzamiento oficial.

Tiempo de implementación

4 – 6 MESES

Considerando las siguientes etapas:

- Análisis de requerimientos.
- Diseño de la aplicación.
- Desarrollo del sistema.
- Integración de métodos de pago.
- Pruebas piloto con la cooperativa.
- Capacitación del personal.
- Implementación y seguimiento.

7. Arquitectura del sistema

El sistema está basado en una arquitectura cliente-servidor compuesta por:

- **Cliente:** aplicación móvil desarrollada en Expo (React Native).
- **Servidor:** API desarrollada en NestJS.
- **Base de datos:** PostgreSQL.
- **Seguridad:** Keycloak y JWT.
- **Contenedores:** Docker.

Flujo del sistema: Usuario → App móvil → API → Base de datos → Respuesta

8. Tecnologías utilizadas

Expo / React Native

TypeScript

NestJS

Node.js

PostgreSQL

Keycloak

Docker

9. Beneficios del sistema

Para estudiantes

- Reducción de tiempos de espera.
- Mayor comodidad.
- Pedidos desde cualquier lugar.

Para la cooperativa

- Mejor organización.
- Control de pedidos.
- Reducción de errores.

Para la universidad

- Modernización de servicios.
- Transformación digital.
- Mejora de imagen institucional.

10. Conclusiones

UTC Pick Sazón representa una solución innovadora que permite modernizar el proceso de compra dentro de la cooperativa escolar de la Universidad Tres Culturas.

El sistema mejora la eficiencia operativa, reduce tiempos de espera y ofrece una experiencia más cómoda para los usuarios.

Desde el punto de vista de la Ingeniería en Sistemas Computacionales, el proyecto integra tecnologías modernas, arquitectura escalable y principios de desarrollo de software que lo convierten en una solución viable y funcional.

PARTE II

Versión extendida

Documentación fuente y detallada del proyecto. Reproduce de forma íntegra los documentos internos que sustentan la propuesta: el **Círculo de innovación** (alcance y reglas de negocio) y el **Algoritmo de ejecución** (implementación técnica).

Anexo A · Círculo de innovación — UTC Pick Sazón

Fuente: [UTC-Proyecto/docs/propuesta/algoritmo-circulo-innovacion.md](#)

Pide fácil, recoge con sabor. Documento fuente del círculo de innovación de la app de cooperativa / dark kitchen escolar UTC, modalidad **Pick Up** (sin envíos). Recorre las fases: **Ocurrencia** → **Idea** → **Propuesta** → **Implementación** → **Valor agregado** → **Adopción**. Aquí viven el alcance y las reglas de negocio.

A.1. Ocurrencia

- Resolver el problema de **congestión** en la cooperativa.
- Implementar **tecnologías** en la cooperativa.
- Crear una **app móvil intuitiva**.
- Agregar una **fila por turnos**.
- **Semáforo de congestión** / pedir para después.
- **Programar tu pedido**.

A.2. Idea

Crear una aplicación web/móvil amigable (p. ej. con Expo Go o framework similar) para una empresa tipo **dark kitchen** que permita pagos con tarjeta. La empresa mantiene un **mínimo y máximo** de productos con la etiqueta “**Preparados**”.

- Pagos con tarjeta mediante plataformas como **PayPal** y **Mercado Pago** (*en esta versión el cobro es simulado; integración real diferida — ver §3.17*).
- La aplicación se enfoca **únicamente en la compra** de productos — **no** es un sistema de envíos.
- Funciona bajo modalidad **Pick Up**: recoger en tienda.
- Cada producto indica un **tiempo de espera** preestablecido.
- **PostgreSQL** para almacenar información y calcular **tiempos promedio** de preparación.
- Campos mínimos indispensables con estructura primaria usando **Primary Key** y **Foreign Key**.
- **Pop-ups** y **notificaciones** sobre el estado de preparación del pedido (*notificaciones locales del sistema, disparadas por la app al detectar el cambio — no push remota con la app cerrada; §3.4*).
- Una vez listo, el pedido debe recogerse en **~10 a 20 minutos**; de lo contrario podrá volver a ofertarse con la tag “Preparados”.
- Estados de producto: *por preparar* (con tiempo estimado de espera) y *preparado* (listo para recoger).
- Mostrar **cuánto tiempo lleva preparado** cada producto.
- **Número de pedido** — cada pedido trae un número secuencial (el primero es **U-00001**, el siguiente U-00002, y así) que es a la vez tu **código de recogida**. Con ese número en tu celular no te acercas

antes de tiempo, evitando la congestión en la ventanilla.

- **Programar tu pedido** — pides con anticipación y recoges a la hora justa, tomando desde antes tu número de pedido.
- **Semáforo de congestión** — un semáforo en vivo detecta cuántos hay en cola: **menos de 5 = Verde**, **de 5 a 10 = Amarillo**, **más de 10 = Rojo** (se detalla en §3.14).

A.3. Propuesta

3.1. Propuesta consolidada

Todo lo que hará UTC Pick Sazón — juntando la idea original con las mejoras que se nos ocurrieron para de verdad acabar con las filas del recreo.

Para el alumno:

- Entrará con su **correo institucional** (`@edu.utc.mx`); si no tiene cuenta deberá crear una. Verá el **menú con fotos**, precios y **cuánto va a esperar** por cada cosa. Lo ya hecho aparece como *Listo para llevar*.
- Arma su pedido, elige cómo pagar (Mercado Pago, PayPal, tarjeta TDC/TDD o **efectivo al recoger**) y lo manda a la cocina sin moverse de su lugar.
- Puede **programar su pedido** para que esté listo justo cuando salga (ej. “lo quiero a las 10:00”).
- Cuando esté listo llega una **notificación** con su **número de pedido**: con ese número no se acerca antes de tiempo, y se acaba el amontonamiento.
- Sigue su pedido en vivo (Pagado → En preparación → Listo → Recogido) y lo recoge mostrando su **código de recogida** (ej. **U-00001**).
- Antes de pedir puede ver el **semáforo de la cooperativa**:  poca fila,  concurrido,  a tope (§3.14).
- ¿No alcanzó a recogerlo? Puede **cancelar** o **extender para después**; si lo deja, el alimento se puede **reofertar** como *Preparado* | *Sin tiempo de espera*.

Para la cooperativa (panel de admin):

- Da de alta y edita el **menú** (productos, precios, fotos, stock mínimo/máximo de “Preparados”).
- Marca los pedidos como **listos**, y eso dispara el **aviso** al alumno (con su número de pedido).
- Ve el **semáforo de congestión en vivo**:  Verde (menos de 5),  Amarillo (de 5 a 10),  Rojo (más de 10) — con el número exacto.
- Aprovecha la **reoferta** / “**Pon tu precio**” para vender lo ya hecho antes de perderlo.
- Ve **qué se vende más y a qué hora pega el pico** en el panel “**Inteligencia del negocio**” (§3.16).
- Trabaja sobre **los mismos pedidos que el alumno**: lo que marca le aparece al alumno en segundos (la app sondea el servidor cada ~15 s y al reenfocar; no es push en tiempo real).
- Maneja su **cuenta**: cerrar sesión, accesibilidad (texto más grande, más contraste) y personalización (sucursal, umbrales del semáforo, horario).

En una frase: **pides desde tu lugar, te avisan con tu turno y recoges sin fila** — y la cooperativa cocina con orden y vende mejor.

3.2. Tecnologías a utilizar

Capa	Tecnologías
Frontend	Expo Go · TypeScript · Tailwind / NativeWind · Arquitectura FSD
Backend	NestJS · TypeScript · Controllers · Services · TypeORM
Base de datos	PostgreSQL · Docker (corre local vía Docker)
Autenticación	Keycloak · Docker (corre local vía Docker)

Cómo se prueba en teléfono (Expo) — versión de prueba solo Android. La app corre en Expo Go (desarrollo) y en web, pero Expo Go SDK 53+ removió las notificaciones del SO; para que salgan se genera un *development build* propio con EAS. Ese build es un **APK de Android** (un iPhone no instala APKs). Para iPhone haría falta un build de iOS por TestFlight, que requiere cuenta de Apple Developer (~\$99/año); alternatively el iPhone puede correr la app por Expo Go (túnel) para ver el flujo, pero sin notificaciones. El túnel funciona igual en Android e iOS — la restricción es de formato/distribución del instalable, no del túnel.

Tablas principales: `users` · `products` · `orders` · `order_items` · `payments` · `preparation_times`.

Seguridad — medidas principales:

- JWT para autenticación
- Roles para permisos
- DTO Validation
- Rate limiting
- MFA para administradores
- Circuit breaker para pagos

3.3. Flujo general del sistema

Usuario → App Expo Go / Panel usuario → Backend NestJS → PostgreSQL
Admin → App Expo Go / Panel Admin → Keycloak (con MFA) → Backend NestJS → PostgreSQL

3.4. Flujo de pedido

```
Usuario inicia sesión
→ selecciona producto (Preparado o con tiempo de espera)
→ revisa tiempo estimado
→ confirma pedido
→ realiza pago
→ backend registra el pedido (le asigna su número secuencial, ej. U-00001)
→ la dark kitchen prepara el producto
→ admin cambia estado a "Listo para recoger" (y se dispara el aviso al alumno)
→ usuario recibe notificación con su número de pedido
→ usuario recoge el pedido con su código
```

3.5. Estados del pedido

Pendiente

En preparación

Listo para recoger

Recogido

No recogido

Cancelado

Listo para recoger después

3.6. Estados del producto

Por preparar

Preparado, listo para recoger

Preparado | Sin tiempo de espera

Calentando tu alimento

No disponible

3.7. Manejo de productos preparados

La cooperativa mantiene un **mínimo y máximo** de productos con etiqueta “Preparados”. Estos productos pueden mostrarse como:

Preparado | Sin tiempo de espera — el producto ya está listo y puede recogerse casi de inmediato.

El sistema también muestra cuánto tiempo lleva preparado cada producto, para que el administrador decida sobre su venta, reoferta o cambio de estado.

Inventario real: “stock de dark kitchen” (D-037)

El `stock` de cada producto son **unidades físicas disponibles ahora** para nuevos pedidos (almacenadas —bebidas, gelatinas— o excedente ya preparado). Es un stock tradicional que el ciclo de vida del pedido mueve solo, con una regla de *dark kitchen*: el **0 NO impide vender**, porque la cooperativa no prepara sin orden y cocina al momento. El candado real de venta es que el producto esté **disponible** (`isAvailable`), no el número de stock.

Momento	Efecto en el stock	Por qué
El admin acepta el pedido (por preparar → preparando)	Aparta lo que haya: <code>stock = máx(0, stock - cantidad)</code>	Reserva del almacén lo disponible; el faltante se cocina al momento (no baja de 0).
Se entrega (recogido)	Sin cambio	Ya se apartó al aceptar; el alumno solo recoge.
No se recoge (por el admin o por vencimiento automático de un pedido listo)	Devuelve como excedente: <code>stock = stock + cantidad</code>	Lo preparado y no reclamado queda reofertable (§3.11).
El alumno cancela un pedido ya listo	Devuelve como excedente: <code>stock = stock + cantidad</code>	El alimento no fue tocado → vuelve a estar disponible.
El alumno cancela un pedido pendiente	Sin cambio	Nunca se apartó.

Ejemplos. *Agua de horchata* (stock 12): se acepta un pedido de 1 → 11; se entrega → 11; si no se recoge → vuelve al 12. *Hamburguesa* (stock 0, se cocina al momento): se acepta → sigue en 0; si no se recoge → 1 (excedente); al reofertarse y venderse → 0, y si no se vende queda 1 para mañana.

Decisión de diseño (honesta): el stock sube solo cuando una unidad preparada queda sin reclamar, no “al empezar a prepararla”. Las transiciones que mueven stock están serializadas con un bloqueo de fila (`FOR UPDATE`) para que no se doble-cuente. El mínimo/máximo siguen siendo etiquetas del admin (`max ≥ min`); no disparan reórdenes automáticas todavía. El vencimiento automático solo alcanza pedidos **listos**; un pedido extendido (`ready_later`) libera su stock por acción del admin o cancelación del cliente.

3.8. Caso: alumno no recoge su pedido

- El sistema considera que la orden estará lista en un máximo aproximado de **10 a 15 minutos**.
- Una vez lista, el alumno tiene un margen aproximado de **10 a 20 minutos** para recogerla.
- Si no la recoge, la app muestra una confirmación:

Tu pedido está listo desde hace varios minutos. ¿Deseas cancelar tu orden o extender tu tiempo para recogerla después? — Opciones: **Cancelar orden** · **Extender para después**

3.9. Si el alumno cancela

El alimento puede volver a ofertarse con la etiqueta **Preparado | Sin tiempo de espera**, porque no fue tocado por el cliente. Si el alimento lleva demasiado tiempo preparado, el indicador pasa a:

Calentando tu alimento — comunica que el alimento ya estaba preparado y solo será calentado para entregarse.

3.10. Si el alumno extiende su tiempo

- Si elige recoger después, el pedido queda marcado como **Listo para recoger después**.
- Podrá recogerlo dentro del horario disponible del mismo día.
- Si al final del día no lo recoge, pasa a **No recogido**. Sobre el dinero: si pagó con tarjeta/online ya está cobrado (pago simulado en esta versión); si eligió efectivo, no se cobró (queda pendiente).

3.11. Reoferta y precio dinámico

Para evitar que los alimentos preparados se queden sin vender, el sistema puede incluir una opción de **reoferta**. Si un producto lleva cierto tiempo preparado, el administrador puede activar una dinámica tipo:

Pon tu precio — o aplicar un descuento controlado.

La finalidad es vender el producto antes de perderlo, aunque sea con menor ganancia.

3.12. Cooperativa por geolocalización

La app trabaja con **varias cooperativas UTC**. No se precarga ninguna: detecta por ubicación la más cercana y la asigna sola, y el pedido se enruta a esa cooperativa.

- Ubicación del usuario con `expo-location` (permiso en primer plano); cooperativa más cercana por distancia, sin default precargado.
- Tocar el encabezado abre la lista para elegir a mano; es el fallback si se niega el permiso. No se puede pedir sin cooperativa.
- El panel del administrador también detecta por geolocalización qué cooperativa opera (no viene fija).
- Cada pedido guarda su cooperativa; la cola del administrador solo trae los de SU cooperativa (`GET /orders/all?branchId=`).

3.13. Menú inicial de la cooperativa

El menú con el que arranca la app refleja los antojos típicos del recreo. Es un punto de partida; la cooperativa puede ajustar productos, precios y disponibilidad:

- **Quesadilla de tinga** — \$38 · se prepara en ~12 min.
- **Combo estudiante** — \$50 · se prepara en ~13 min.
- **Hamburguesa de la casa** — \$65 · se prepara en ~15 min.
- **Papas con queso** — \$32 · se prepara en ~8 min.
- **Agua de jamaica** — \$18 · lista para llevar.
- **Boneless BBQ** — \$58 · se prepara en ~14 min.
- **Papas a la francesa** — \$28 · se prepara en ~7 min.

- **Esquites en vaso** — \$22 · se prepara en ~6 min.
- **Gelatina de mosaico** — \$15 · lista para llevar.
- **Agua de horchata** — \$18 · lista para llevar.

Las aguas frescas y la gelatina suelen ofrecerse como *Preparado* | *Sin tiempo de espera*; el resto se prepara al momento mostrando su tiempo de espera.

3.14. Semáforo de congestión

- **Qué cuenta:** los pedidos en cola ahorita — *pendientes, en preparación o listos esperando*. No cuentan los recogidos, cancelados ni los dejados para después.
- **Colores:** 🟢 Verde = menos de 5 · 🟡 Amarillo = de 5 a 10 · 🔴 Rojo = más de 10. Son números ajustables.
- **Quién lo ve:** el alumno ve el color (con un texto tipo “tranquila / concurrida / llena”); el admin ve el número exacto además del color.
- **Sin trampas:** se calcula con la hora del servidor, no el teléfono.

3.15. Pedido programado

- **Con tiempo:** se programa con mínimo 30 minutos de anticipación.
- **Para hoy:** una hora del mismo día, dentro del horario de la cooperativa.
- **La cocina no se adelanta ni se atrasa:** el sistema calcula cuándo empezar y le avisa al negocio justo cuando toca; en el panel sube de prioridad con un “🕒 Empezar ahora”.
- **No infla el semáforo antes de tiempo:** no cuenta en la cola hasta que se abre su ventana (~20 min antes de la recogida).
- Trae su número (ej. U-00001) y se sigue en vivo hasta recogerlo.

3.16. Inteligencia del negocio

En el panel del administrador hay una tarjeta “**Inteligencia del negocio**” con dos datos calculados por el servidor sobre pedidos reales:

- **Producto más vendido:** el que más unidades ha vendido (no cuenta cancelados).
- **Hora pico:** la franja horaria con más pedidos, en hora local de la cooperativa (no cuenta cancelados).
- **Quién lo ve:** solo el administrador.

Es distinto del semáforo (§3.14, congestión ahorita) y de los tiempos promedio de preparación (§3.7): esto es la demanda histórica.

3.17. Métodos de pago

Al pagar, el alumno elige entre **cinco** métodos: Mercado Pago, PayPal, TDC, TDD y Efectivo al recoger.

Estado actual (honesto): el cobro con tarjeta/online es **SIMULADO**; no hay integración real con Mercado Pago, PayPal ni un procesador de tarjeta.

- **Tarjeta (TDC/TDD):** formulario real; solo se guardan los últimos 4 dígitos. La aprobación se simula, protegida por un circuit breaker. Se pueden guardar en “Mi cartera”.
- **Mercado Pago / PayPal:** se registran como método; su cobro real queda diferido.
- **Efectivo al recoger:** el único real — se cobra en el mostrador al entregar.

La integración con pasarelas reales queda fuera del alcance de esta demo (ver `decisiones.md` D-006 / D-033).

A.4. Implementación

El algoritmo para construir todo lo de la Propuesta vive en el **Anexo B (Algoritmo de ejecución)**. El detalle técnico está en `architecture-propuesta.md` y las decisiones en `decisiones.md`.

Estado de implementación (2026-07-02). Todo lo descrito está implementado y verificado. En el último cierre (decisiones D-027...D-034) se completaron: buscador de menú, tiempos promedio de preparación, semáforo ajustable desde el admin, métricas de más-vendido y hora pico, auto-vencimiento de la ventana de recogida, estado “Calentando”, accesibilidad funcional, sucursal persistida con el pedido y MFA de admin forzada. Los pagos usan un formulario real de tarjeta con aprobación simulada protegida por circuit breaker; las pasarelas reales quedan diferidas.

A.5. Valor agregado

- **Mata la fila del recreo de verdad:** turnos + semáforo + pedido programado reparten la llegada de cientos de alumnos.
- **Nadie se queda viendo a ciegas:** siempre sabes cuánto falta y cuándo está listo.
- **Incluye a todos:** pago en efectivo al recoger (y a futuro, monedero escolar) y opciones de accesibilidad.
- **La cooperativa gana:** menos desperdicio (reoferta), mejores compras (datos) y operación más tranquila en la hora pico.
- **Escala:** el mismo diseño sirve para otros planteles (selección de sucursal por cercanía).

Ideas en el tintero (roadmap): monedero/saldo escolar, alérgenos e ingredientes, favoritos (“lo de siempre”), avisos del día y tope de pedidos por horario.

A.6. Adopción

- Pruebas con alumnos reales en el editor y en el teléfono (Expo Go), iterando lo que confunda.
- Arranque en una sola cooperativa (la de la UTC) con el menú actual.
- Demo a los dirigentes mostrando cómo baja la congestión y cómo se vende lo preparado.
- Capacitación corta al personal para el panel de admin.
- Si convence, se amplía: más horarios, más productos y —si se da— más planteles.

¿Cuándo fue un éxito? Se da un periodo de adopción de 1 mes: si una buena parte de los alumnos ya pide por la app y baja la congestión del recreo, se considera adoptada y se amplía.

Anexo B · Algoritmo de ejecución — UTC Pick Sazón

Fuente: UTC-Proyecto/docs/propuesta/Algoritmo-ejecucion.md

Fase de **Implementación** del círculo de innovación: el algoritmo concreto para construir todo lo que vive en la Propuesta (Anexo A, §3).

B.0. Planteamiento

Objetivo. Resolver el problema de congestión en la cooperativa escolar, principalmente en horarios donde los alumnos salen a desayunar; la solución debe poder aplicarse en cualquier modalidad (Escolarizada | Ejecutiva), horario y sucursal o plantel.

¿Para quién va dirigido? Para alumnos, maestros, personal escolar y cualquier persona que desee comprar en la cooperativa, y para los dirigentes de la cooperativa para facilitar la gestión de la congestión.

¿Cómo se va a dirigir? A través de arquitecturas definidas y tecnologías concretas que permitirán la creación de una app móvil.

Nota. Se estarán haciendo constantes pruebas en el editor de texto elegido y en el framework a trabajar.

B.1. Algoritmo de ejecución (pasos)

1 · Definir el alcance

A través de los 3 primeros flujos del proceso de innovación: **Ocurrencia, Idea y Propuesta** (alcance, usuarios, auth, funcionalidades, tecnologías, arquitectura, distribución). Con la propuesta lista se genera el diseño de las pantallas.

2 · Definir diseño y paleta de colores

Variante	Hex
Azul institucional	#021E5E
Azul alternativo	#0A2E7A
Naranja principal	#E34100
Naranja alternativo	#F26336

Se crean dos diseños según el perfil: (1) pantalla de **Colaborador** de la cooperativa (dark kitchen) y (2) pantalla de **Usuario** con propósito de usar la plataforma.

3 · Estilos base y levantar servicios

Definir Welcome Page, Login Page, Logo, Nombre y Eslogan. Luego levantar los servicios: **Node.js**, **Docker**, puertos 5432 (PostgreSQL) y 8080 (Keycloak). Se construyen las imágenes de PostgreSQL y Keycloak.

4 · Distribución de carpetas

```
UTC-Proyecto/  
  frontend/  (la app Expo: pantallas del alumno)  
  backend/   (NestJS: la lógica y la conexión a la base de datos)  
  infra/     (Docker: PostgreSQL + Keycloak)  
  brand/     (másters del logo; el sistema de diseño vive en frontend/src/shared/theme +  
  ui)  
  docs/      (este documento, la propuesta y las decisiones)
```

5 · Construir las pantallas del alumno (cliente)

- Welcome y Login con el correo institucional (@edu.utc.mx).
- Inicio: menú con fotos, precios y tiempo de espera, más el carril de “listos para llevar ya” y el buscador.
- Detalle del producto, carrito y checkout (Mercado Pago, PayPal, tarjeta o efectivo al recoger).
- Seguimiento del pedido (código de recogida tipo U-00001) y pestañas de Pedidos y Perfil.

6 · Construir el panel del administrador

- Alta y edición del menú (productos, precios, fotos y stock de “Preparados”); el stock además se mueve solo con el ciclo del pedido (§18).
- Recibir pedidos y marcarlos como “Listo” (dispara el aviso al alumno).
- Ver el semáforo de congestión en vivo (§12).
- Manejar la reoferta / “Pon tu precio”.
- Cerrar sesión y ajustar su cuenta: accesibilidad y personalización funcional (nombre/horario y umbrales del semáforo).
- Los pedidos son los mismos para admin y alumno: lo que el admin marca aparece al alumno en segundos (sondeo cada ~15 s; no es push).

7 · Conectar con la base de datos

- Guardar productos, pedidos, pagos y tiempos en PostgreSQL (6 tablas).
- El backend valida quién es quién (alumno o admin) con Keycloak + JWT antes de dejar hacer nada.

8 · Funciones que descongestionan

Número de pedido secuencial (U-00001), pedido programado (§13) y el semáforo, más las notificaciones de “tu pedido está listo” (avisos locales del sistema que dispara la app al detectar el cambio; no push remota con la app cerrada).

9 · Probar y desplegar

Probar con alumnos reales, corregir lo que confunda y dejar todo listo para la demo. Tras el arranque, periodo de adopción de 1 mes.

10 · Cooperativa por geolocalización

Varias cooperativas UTC; no precarga ninguna: detecta la más cercana (`expo-location`) y enruta el pedido a esa cooperativa. Se puede elegir a mano tocando el encabezado (fallback si se niega el permiso). La cola del admin solo trae los pedidos de SU cooperativa (`GET /orders/all?branchId=`).

11 · Menú inicial

Menú base para el recreo, editable por la cooperativa: Quesadilla de tinga \$38, Combo estudiante \$50, Hamburguesa de la casa \$65, Papas con queso \$32, Agua de jamaica \$18, Boneless BBQ \$58, Papas a la francesa \$28, Esquites en vaso \$22, Gelatina de mosaico \$15, Agua de horchata \$18. Aguas y gelatina suelen estar listas para llevar; los demás se preparan al momento.

12 · Semáforo de congestión (parámetros y visibilidad)

Lo calcula el backend como el número de pedidos en cola (`pending` , `preparing` , `ready`); no cuentan `ready_later` , `picked_up` , `not_picked_up` ni `cancelled` . Umbrales ajustables (`UMBRAL_AMARILLO = 5` , `UMBRAL_ROJO = 10`): Verde $n < 5$, Amarillo $5 \leq n \leq 10$, Rojo $n > 10$. El admin ve el conteo exacto; el cliente ve color + etiqueta. Usa la hora del servidor.

13 · Pedido programado

- Mínimo 30 minutos de anticipación y para el mismo día; si no cumple, se rechaza.
- Hora de empezar a preparar = hora de recogida – tiempo de preparación estimado.
- Al llegar esa hora se avisa al negocio; en la cola sube de prioridad con “Empezar ahora”.
- Entra al semáforo solo cuando se abre su ventana (~20 min antes), no antes.
- Lleva su número secuencial (U-00001) y se sigue en vivo.

14 · Número de pedido secuencial

Cada pedido recibe un número correlativo legible al crearse (U-00001, U-00002...). Es el código de recogida del cliente y el identificador en la cola del admin. Lo genera la base de datos, único y en orden.

15 · Inteligencia del negocio

Tarjeta en el dashboard del admin con dos indicadores del servidor (solo admin): **producto más vendido** (más unidades, excluye cancelados) y **hora pico** (franja con más pedidos, hora local, excluye cancelados). Es la demanda histórica, distinta del semáforo y de los tiempos promedio.

16 · Cierre de la propuesta (estado 2026-07-02)

Se completaron los detalles (D-027...D-034): buscador funcional; estado “Calentando” y “preparado hace X min”; tiempo promedio de preparación (últimas 20 muestras); semáforo ajustable guardado en el servidor; métricas de negocio; auto-vencimiento de la ventana; accesibilidad real; foto editable por URL;

sucursal que viaja con el pedido; MFA (TOTP) obligatoria para admin; pago con tarjeta con formulario real (guarda solo los últimos 4) y aprobación simulada protegida por circuit breaker (valida formato 13–19 dígitos, no el checksum de Luhn).

17 · Métodos de pago (qué es real)

Cinco métodos: Mercado Pago, PayPal, TDC, TDD y Efectivo al recoger. **Honesto:** el cobro con tarjeta/online es SIMULADO. Tarjeta: formulario real, guarda solo los últimos 4, aprobación simulada + circuit breaker. Mercado Pago/PayPal: se registran, cobro real diferido. Efectivo: el único real, se cobra en el mostrador. Las pasarelas reales quedan fuera del alcance (D-006/D-033).

18 · Inventario real — “stock de dark kitchen” (D-037)

El stock son unidades físicas disponibles ahora (almacenadas o excedente preparado) y el ciclo del pedido lo mueve solo, en el servidor y transaccional (nunca baja de 0). El 0 no impide vender; el candado es `isAvailable`. Al aceptar: aparta lo que haya (`máx(0, stock - cantidad)`). Al entregar: sin cambio. No recogido / cancelación de un pedido ya listo: devuelve como excedente (`stock + cantidad`). Cancelar un pedido pendiente: sin cambio. Las transiciones se serializan con bloqueo de fila (`FOR UPDATE`).

B.2. Implementación técnica (2026-06-24)

19.1 Infraestructura

- **Orquestación:** Docker + Docker Compose. PostgreSQL 16 y Keycloak 26 como servicios containerizados en desarrollo local.
- **Base de datos:** PostgreSQL 16. Esquema: 6 tablas (`user_profile`, `products`, `orders`, `order_items`, `payments`, `preparation_times`) + 5 enumerados. Migraciones TypeORM (`synchronize: false`); datos de demo por `seed-demo.sql`.
- **Identidad:** Keycloak 26, realm `utc-food`, roles `admin` y `user`. Admin con MFA (TOTP); cliente con auto-registro restringido a `@edu.utc.mx`.

19.2 Frontend — Expo + React Native

Stack: Expo SDK 56 (managed), React Native 0.85.3, React 19.2.3, TypeScript. **Arquitectura:** Feature-Sliced Design (`app/`, `pages/`, `widgets/`, `features/`, `entities/`, `shared/`). **Librerías:** React Navigation 7, Zustand, expo-font, lucide-react-native, safe-area-context. **Identidad visual:** tipografías de marca (Bricolage Grotesque, Plus Jakarta Sans, Space Mono) por `expo-font`; tokens en `shared/theme/tokens.ts`.

```
cd frontend
npm install
npx expo start      # inicia Metro; QR para Expo Go o "w" para navegador
```

19.3 Backend — NestJS + TypeORM

Stack: NestJS 11, TypeORM, TypeScript. **Arquitectura:** Clean Architecture (`presentation/`, `application/`, `infrastructure/`, `domain/`). **Seguridad:** `class-validator` + `ValidationPipe` global; JWT de Keycloak validado con `passport-jwt` + `jwt-keycloak` (RS256); guards `JwtAuthGuard` + `RolesGuard`; rate limiting (5 req/min en auth, 60 default); MFA para admin.

Rutas principales: `POST /auth/register`, `POST /auth/login`, `POST /auth/admin/login` (exige TOTP), `GET /auth/me`, `GET /health`, y rutas CRUD (productos, órdenes) protegidas por rol.

```
cd backend
npm install
cp .env.example .env
npm run migration:run      # aplicar esquema (idempotente)
npm run start:dev          # inicia en puerto 3002 (watch mode)
```

19.4 Orquestación local (desarrollo)

1. **Infraestructura:** `cd infra && docker compose up -d && ./keycloak/seed-admin.sh`
2. **Backend:** `cd backend && npm install && npm run migration:run && npm run start:dev` (puerto 3002)
3. **Frontend:** `cd frontend && npm install && npx expo start`

Invariantes: `infra/.env` y `backend/.env` consistentes en `DB_*` y `KEYCLOAK_*`; `KEYCLOAK_BACKEND_CLIENT_SECRET` = `BACKEND_CLIENT_SECRET`; backend escucha en el puerto 3002.

19.5 Estructura del repositorio

```
UTC-Proyecto/
├─ frontend/      - Expo + React Native
├─ backend/       - NestJS + TypeORM
├─ infra/        - Docker Compose, inicialización
├─ brand/        - Activos de marca (logo HQ)
├─ docs/         - Documentación por temas
├─ propuesta/    - círculo de innovación, ejecución
├─ arquitectura/ - especificación técnica, decisiones
├─ datos/        - esquema, datasets de demo
├─ operacion/    - runbooks
├─ historico/    - regresiones, reportes
└─ superpowers/  - plans, specs, rules
```

Gestión de secretos: `.env` (valores reales), `node_modules/`, `dist/` y `pgdata/` en `.gitignore`. Solo se versionan los `*.env.example`.

19.6 Distribución de la aplicación

Desarrollo: Expo Go (QR o USB+adb). **APK Android:** `npx expo prebuild --platform android` y `npx eas build --platform android` (requiere cuenta EAS); alternativa local `./gradlew assembleDebug`.

Producción: Google Play (fuera de alcance). **Web:** `npx expo start --web` (aproximación funcional).

19.7 Verificación pre-commit

- **Frontend:** `npx tsc --noEmit`; `npx expo export --platform android`.
- **Backend:** `npm run build`; `npm run lint`.
- **Integración:** `curl http://localhost:3002/health` → `{status, db}`; login cliente/admin → tokens válidos.

Puerto del backend: 3002 (especificado en `backend/.env`; `PORT=3002`).